FinPilot

Feature Guide

How every feature we built works: data, algorithms, tech stack, and why each choice was made.

Covers: Opt-out simulator · Phone OTP for Google Pay link · Forecast & Risk · Photo prices · Statement OCR · FinPilot Pay phone app · Payment security review · Phone pairing & live sync · Read cache · Login and onboarding flow

Written from the code in this repository. Demo build: no real money moves and no bank is contacted.

Contents

1. System overview and architecture
2. Tech stack and why
3. Datasets and data model
4. Feature 1: Opt-out expense simulator
5. Feature 2: Phone OTP for the Google Pay link
6. Feature 3: Forecast & Risk
7. Feature 4: Photo prices
8. Feature 5: Statement and receipt OCR
9. Feature 6: FinPilot Pay phone app
10. Feature 7: Automatic payment security review
11. Feature 8: Phone pairing and live sync
12. Feature 9: Read cache with write-through
13. Feature 10: Login, onboarding and dashboard changes
14. Security, privacy and data integrity
15. API reference
16. Testing, running it, and known limits

> **How to read this guide.** Each feature has the same shape: what it does, a diagram, how it works step by step, the data it uses, the stack and the reason for it, and its limits. Numbers quoted (thresholds, sizes, seeds) are the actual constants in the code.

1. System overview

One ledger per user, many views of it.

FinPilot is a personal-finance copilot. Everything revolves around one **ledger per user** (accounts, transactions, budgets, goals). The web app, the phone app and every analysis read and write that same ledger, which is why a payment made on the phone shows up on the dashboard within about a second.

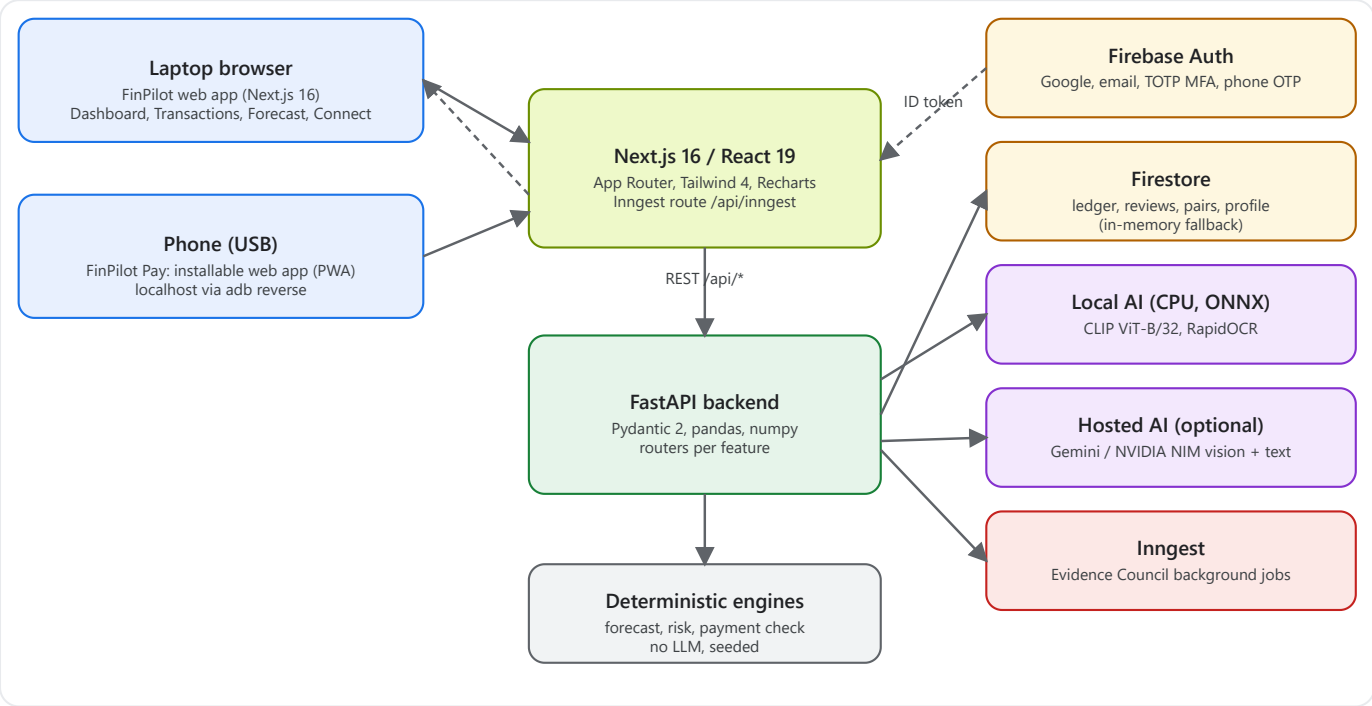


Figure 1. Components and how they talk. Solid arrows are requests; dashed arrows are identity or fallback.

The three rules the design follows

- **Numbers are computed, not generated.** Forecasts, risk, simulators and the payment check are deterministic code over your data. LLMs are used only for language and vision (naming a photographed item, answering chat), never to produce a financial figure.
- **Private data stays local first.** OCR and image embeddings run on the machine (ONNX on CPU). Statements are redacted before anything is previewed or stored. Hosted models are optional fallbacks.
- **One source of truth.** There is a single backend store module. Every router goes through it, so caching, invalidation and the live-sync counter are consistent.

Request path of a typical page

Step	What happens
1	Browser holds a Firebase ID token (or a demo/guest id). `apiFetch` adds it as `Authorization: Bearer`.
2	FastAPI dependency `resolve_user_id` checks the token's subject equals the `{user_id}` in the URL. Demo personas and guest sandboxes are public fixtures and need no token.
3	Router reads rows through `store.query` (cache, then Firestore) and computes the answer with pandas/numpy.
4	Response is a Pydantic model, so the frontend types in `lib/types.ts` mirror it.
5	`LiveSync` polls a change counter; when it moves, the API cache is cleared and every mounted page refetches.

2. Tech stack and why

Each choice, and the alternative we did not take.

Layer	Choice	Why this	Instead of
Web UI	Next.js 16 (App Router), React 19	File-based routes, server/client split, one codebase for the site and the installable phone app	Separate native app (Flutter / React Native)
Styling	Tailwind CSS 4 + shadcn/Radix primitives	Fast to build accessible, consistent components; design tokens (lime / sage / ink)	Hand-written CSS
Charts	Recharts	React-native charting; fan charts, bars, calibration plots with little code	D3 (more code), Chart.js
Animation	Framer Motion, NumberFlow	Smooth numeric and layout transitions	CSS-only
Backend	FastAPI + Pydantic 2	Typed request/response, auto validation, quick to test; Python owns the numeric stack	Node API (would split the ML/finance code)
Numerics	pandas 2.2, numpy 2.2	Monte Carlo, group-bys and rolling statistics are one-liners and fast for months of data	Pure Python loops
Database	Firestore (Admin SDK), memory fallback	Serverless, per-user documents, free tier, same project as Auth; fallback makes tests and demos work offline	SQL (needs hosting and migrations)
Auth	Firebase Auth (Google, email, TOTP MFA, phone OTP)	Hosted, secure, phone SMS and reCAPTCHA handled for us	Own password and OTP system
Image embeddings	CLIP ViT-B/32 via fastembed (ONNX)	No torch install, CPU only, zero-shot: needs no training set for new items	Training a classifier (needs labelled data)
OCR	RapidOCR (PP-OCR, ONNX), incl. Devanagari	Runs locally, no cloud upload of bank statements, reads Hindi and Marathi	Cloud OCR (privacy cost)
Vision / text LLM	Gemini, NVIDIA NIM (optional)	Handles odd bills and unrecognised items; failover between providers	One vendor
Background jobs	Inngest	Parallel agents, retries and a run dashboard for the Evidence Council without us building a queue	Celery/Redis
Phone app	PWA (web manifest, standalone)	Same React code installs without the browser bar; secure context over `adb reverse` unlocks camera	Native Android build
QR / UPI	`qrcode` lib, Chrome `BarcodeDetector`, `upi://pay` URI	Standard UPI deep-link format; built-in scanner needs no extra library	Third-party scanner SDK
Tests	pytest (backend), tsc + eslint (frontend)	Deterministic engines are easy to assert on	Manual only

3. Datasets and data model

Where every number in a demo comes from, and how we keep that honest.

There is **no external dataset download** for the financial features. Data comes from five sources, and each feature states which one it uses.

Source	What it is	Used by
Persona seeds	`ingest/personas.py` generates 12 months of transactions for three demo users (Priya, Arjun, Meera): salary, rent, EMIs, subscriptions, SIPs with jitter so recurring-detection has to find the pattern, a discretionary profile per category and planted anomalies.	Dashboard, Transactions, security review, simulator (detected subscriptions)
Synthetic variable-income history	`plan/synthetic.py`: seeded 48-month income / essential / discretionary series for three profiles (stable, moderate, volatile). Test months use a separate seed so tuning never touches them.	Forecast & Risk when you have under 4 months of real data, or by choice
Your own ledger	Rows you created: UPI payments from the phone, imported statements, linked connectors, receipts.	Everything, and it always wins over priors
Catalog priors	`expenses/catalog.py`: everyday items with a CLIP description, category, aliases and typical INR price, adjusted by a city factor.	Photo prices (first-time items only)
Merchant rules	`ingest/categorize.py`: ordered regex list of Indian merchants to a category; anything unknown is `other`.	Phone payments, imports

Collections (Firestore) added or used by these features

Collection	Key fields	Purpose
transactions	id, user_id, date, amount (negative = spend), merchant, category, description, review_status, created_at	The ledger. UPI payments have description `UPI to <id>`
accounts	id, user_id, name, type, balance	Balance is decremented on each payment
payment_reviews	id (= transaction id), verdict, heavy, status (open/confirmed/reported), payload JSON	Security review of heavy or unusual payments
pay_pairs	id (= pairing code), user_id, created_at, claimed_at	Lets the phone write to a signed-in user's ledger
price_memory	user_id, item, price, photo embedding	What you taught Photo prices
profiles	id, level, answers	Onboarding questionnaire

> **Honesty rule.** Forecast & Risk labels its data source on every screen: when the series is synthetic the report says so in bold. No synthetic number is ever presented as yours.

Feature 1

Opt-out expense simulator

See what you would save, per year and in a 12% SIP, if you cancel everyday subscriptions and habits.

What it does

A tab on **/simulate**. You list recurring spends (Spotify, Netflix, Hotstar, doodhwala, newspaper, gym, broadband and your own). A toggle cancels each one, and the page shows monthly saving, new savings rate, yearly saving and the future value if the saved money went into a SIP.

How it works

- Each item has `cost` (today) and `replacement` (what you would still spend, e.g. packet milk instead of a doodhwala). Saving = cost - replacement for cancelled items.
- Before: `savings = income - other - sum(cost)`. After: `savings + saving`. Savings rate = savings / income.
- 12-month future value of the monthly saving uses the annuity-due formula with `SIP_RATE = 12%` p.a.: `FV = P * ((1+r)^n - 1) / r * (1+r)`, r = 12%/12.
- Subscriptions detected in your real data (the dashboard's recurring obligations of category *subscriptions*) are offered as one-click additions, normalised to a monthly amount by frequency (weekly 52/12, quarterly 1/3, annual 1/12).

Formula	Value
Monthly saving	sum over cancelled (cost - replacement)
Future value	P x ((1+r)^12 - 1)/r x (1+r), r = 0.01
Savings rate	(income - other - spend) / income

Data and stack

DATA
your inputs + detected subscriptions from `/api/dashboard`

STACK
React state + `useMemo`, Recharts area chart, Framer Motion

WHY CLIENT-SIDE
instant feedback while toggling; no server round trip for pure arithmetic

NO LLM
all arithmetic

Limits

The 12% return is illustrative, not a promise, and is labelled as such. Items you add are not saved between sessions.

Feature 2

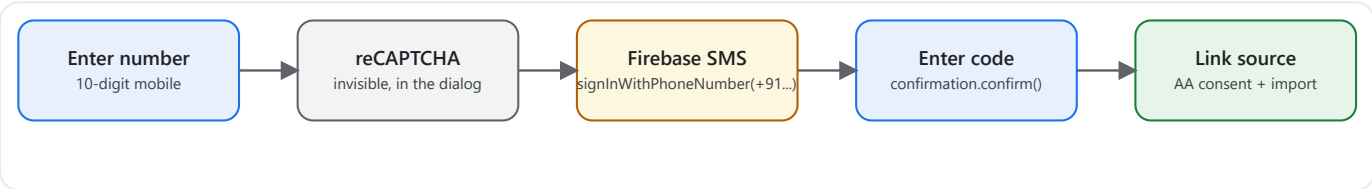
Phone OTP for the Google Pay link

Prove you own the mobile number before linking a UPI source.

What it does

In **Connect > Google Pay**, after choosing a demo profile and entering a mobile number, the dialog sends a real SMS one-time code through Firebase Phone Auth and asks for it before the Account Aggregator consent step proceeds.

How it works



- The OTP uses a **separate Firebase app instance** named `phone-verify` with in-memory persistence. Verifying a number therefore never signs the user in as that number or changes who the app thinks the user is.
- A new `RecaptchaVerifier` is created for every attempt and cleared after failure or success, because a verifier cannot be reused.
- Numbers are normalised to `+91` and validated as 10 digits before any SMS is sent.

Data and stack

PROVIDER
Firebase Authentication, Phone provider

CLIENT
`lib/firebase/auth.ts` : sendPhoneOtp, confirmPhoneOtp

UI
`LinkGooglePayDialog` phases: details, verify, otp, working

WHY FIREBASE
SMS delivery, rate limiting and abuse protection are hard to do safely ourselves

Limits

Needs the Phone provider enabled in the Firebase console and an authorised domain (localhost is allowed). Real SMS costs apply beyond the free tier; test numbers can be configured in Firebase.

Feature 3

Forecast & Risk

A probabilistic cash-flow forecast, the chance you run short, four scenarios side by side, and proof of how accurate it has been.

What it does

The **/forecast-risk** page has four tabs: **Forecast** (P10-P90 fan of income, essentials, discretionary and net cash), **Risk** (probability of falling below a safety reserve each month, severity, drivers), **Scenarios** (do nothing vs cut discretionary vs extra debt payment vs extra saving, on the same simulated futures) and **Accuracy** (backtest versus baselines and calibration of the risk numbers).

Pipeline

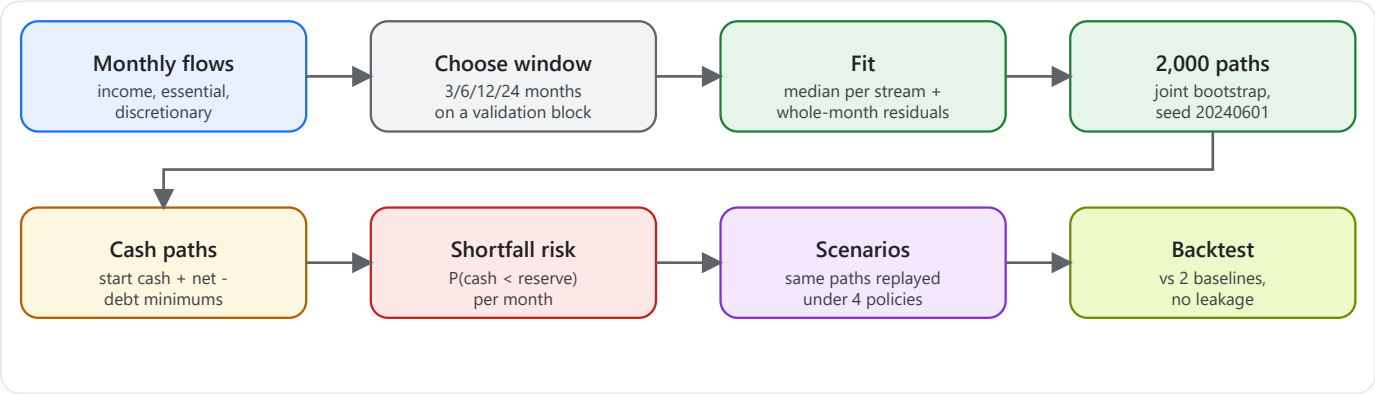


Figure 2. One request builds the whole report. Everything after the history is a pure function: same inputs and seed give identical output.

The forecaster (<code>plan/forecast.py</code>)

- **Streams.** Three monthly series: income, essential spend, discretionary spend.
- **Point forecast.** Median of the last *W* months per stream. *W* is chosen from (3, 6, 12, 24) by one-step-ahead rolling-origin error of net cash on a *validation block that sits before the test months*. The longest window wins unless a shorter one beats it by more than 5%, because the validation block is short and noisy.
- **Uncertainty.** The residual vector of a *whole month* (all three streams together) is resampled with replacement. Resampling jointly keeps correlation: a bonus month and a heavy-spend month stay linked.
- **Paths.** `N_PATHS = 2000` simulated futures, `SEED = 20240601`, horizons 3, 6 or 12 months. Bands are empirical P10 / P50 / P90.

Risk (<code>plan/risk.py</code>)

- Cash on each path = starting cash + cumulative (income - essential - discretionary) - debt minimums.
- **Shortfall probability** for a month = share of the 2,000 paths where cash is below the reserve (default: median essential spend over the window).
- Level low / moderate / high compares that probability with your tolerance. Severity (how deep the shortfall goes) and ranked *factors* are computed from the same paths. Explanatory sentences are templated from the numbers, not generated by an LLM.

Scenarios (<code>plan/compare.py</code>)

Four fixed monthly schedules (do nothing, cut discretionary, pay extra on debt, save extra) are replayed over the **same** 2,000 paths, so differences are policy effects and not simulation noise. Cash arithmetic reuses the planner's

`DebtBook`, so "cash" has one definition across the app. A recommendation block states the trade-offs.

Accuracy (<code>plan/accuracy.py</code>)

- **No leakage:** every forecast origin refits on months strictly before it; the window is tuned on validation; test months are only used for scoring.
- **Baselines:** historical average and recurring-payment forecasts. Point error (MAE) and interval coverage/width are compared.
- **Calibration:** predicted shortfall probabilities are binned and compared with how often shortfalls actually happened.

Data and stack

REAL DATA your monthly flows, needs at least `MIN_MONTHS = 4`	**SYNTHETIC DATA** 48 months, 3 profiles, shown with a disclosure banner
STACK numpy, pandas, Pydantic; Recharts on the frontend	**ENDPOINT** `GET /api/forecast-risk/{user}` with horizon, sims, seed, profile

Why a bootstrap and not ARIMA/Prophet? Personal finance gives 6-24 noisy months. A model with many parameters overfits; the median-plus-resampled-residuals approach has one tuned parameter (the window), keeps the real shape of your variability, and is explainable in one sentence.

Tests and limits

19 pytest cases cover determinism, window selection, leakage guard, risk monotonicity and scenario paths. Limits: a long upward trend or a one-off life event is not in the history, so the fan will not anticipate it; the report states assumptions on screen.

Feature 4

Photo prices

Take or choose a photo, and the app says what it looks like and suggests a price. Confirm once and it remembers.

What it does

On **/photos**, tapping the button first asks *camera or upload*. The photo is analysed and the result reads "Looks like X" with a price field. Confirming the name and price saves it so next time the same item is priced from your own history.

Pipeline

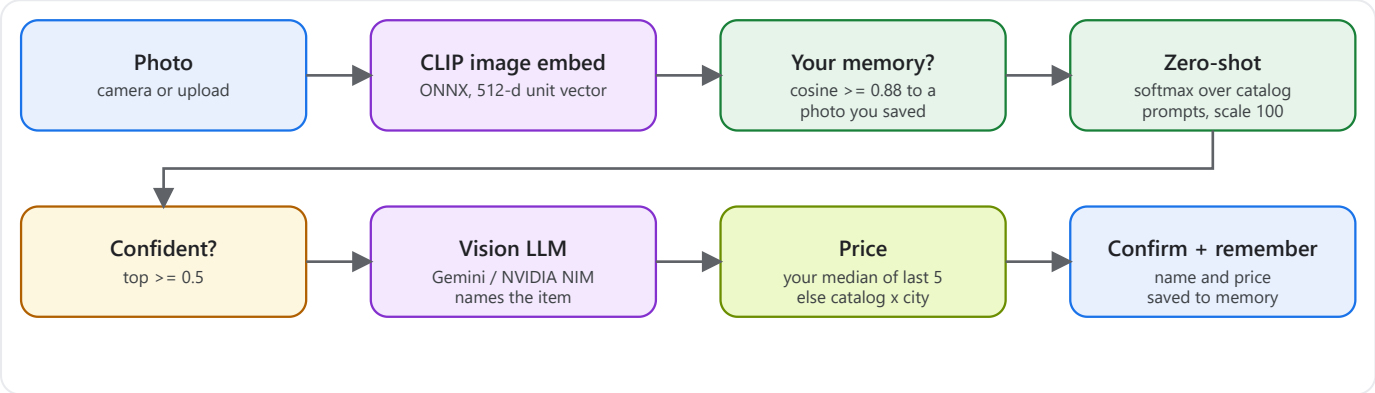


Figure 3. The order is deliberate: your own history first, then a zero-shot model, then a language model only if needed.

How each stage works

- **Embedding.** The image goes through CLIP ViT-B/32 (ONNX via fastembed, CPU) to a 512-dimension unit vector. Models download once (~600 MB).
- **Memory.** If the cosine similarity to a photo you saved earlier is at least `0.88`, the item is the same thing as before and its price comes from your history.
- **Zero-shot.** Otherwise the vector is compared with CLIP text embeddings of "*a photo of <catalog prompt>*" for each catalog item. Logits are scaled by 100 and passed through a softmax to give confidences. **No training happens**: the catalog is just text.
- **Confidence gate.** Under `0.5` the price is left blank rather than guessed, and the vision LLM (Gemini if a key is set, else NVIDIA NIM) names the item.
- **Price.** Your median of the last 5 prices for the item, else the catalog price times a city factor. The result shows the range of your own history.

Datasets

TRAINING SET
none: zero-shot CLIP, plus your saved items

CATALOG
`expenses/catalog.py`: item, prompt, aliases, category, INR price prior

MEMORY
`PriceMemoryRow`: item, price, embedding

ENDPOINTS
`POST /expenses/{u}/snap`, `/suggest`, `/teach`, `GET /items`

Why this stack

CLIP gives usable recognition for hundreds of everyday objects without collecting or labelling images. ONNX through fastembed avoids installing PyTorch. Keeping the LLM as the last resort keeps cost, latency and privacy low, and the user's own confirmed prices always outrank any prior.

Limits

Prices are priors, not market quotes. Visually similar items (two bottle sizes) can be confused; that is why the user confirms the name and price once.

Feature 5

Statement and receipt OCR

Photograph a bank statement, passbook or receipt and get transactions, with personal identifiers removed first.

What it does

The **Upload document** page and the statement flow read a photo or scan, show a redacted preview, and import the parsed rows into the ledger after you confirm.

Pipeline

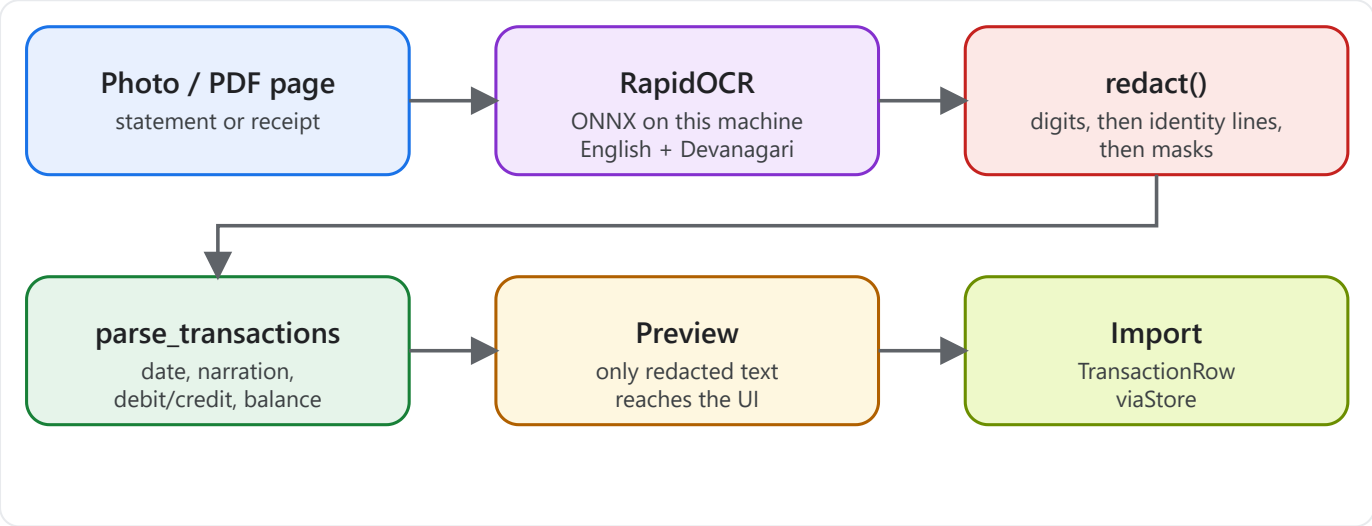


Figure 4. Redaction is the second step, so nothing downstream ever sees account or identity data.

How it works

- **OCR.** RapidOCR (PP-OCR on ONNX) runs on this machine. A Devanagari recogniser is tried first (it also reads Latin letters and digits); if fewer than 3 Devanagari letters are found, the English model is used. Boxes are sorted into reading order and merged into lines by vertical position.
- **Redaction (`statement_ocr.redact`).** Devanagari digits are converted to ASCII; whole lines carrying identity (account holder, customer ID, name, address, branch, nominee, in English, Hindi and Marathi) are dropped; then ordered masks replace email, UPI IDs, PAN, Aadhaar (before account numbers so a 12-digit Aadhaar is not mislabeled), IFSC, phone numbers and long account numbers. The response reports how many of each kind were removed.
- **Parsing.** `parse_transactions` extracts date, narration and debit/credit amounts per line; `parse_receipt_txn` handles single receipts. Rows are categorised with the same merchant rules as everything else.

Data and stack

INPUT
image bytes (PNG/JPEG), Hindi / Marathi / English

OUTPUT
plain dicts, then `TransactionRow` on import

ENDPOINTS
`POST /api/ocr/extract/{u}` then `/import/{u}`

WHY LOCAL
bank statements are the most sensitive file a user has; no cloud upload is needed

Limits

OCR quality depends on the photo. Unusual layouts may need the hosted vision model; the preview step lets the user catch errors before import.

Feature 6

FinPilot Pay: the phone payment app

A Google-Pay-style scan-and-pay web app installed on the phone. No real money moves; every payment is booked into the real ledger.

What it does

Open `/pay` on the phone (USB-connected). You scan any UPI QR (or pick a person, type a UPI ID), enter the amount, choose the account, enter any 4-6 digit PIN, and see a success screen with the UPI transaction id. The same payment appears on the website's dashboard and Transactions page.

Screens

Screen	Behaviour
Home	search pill and avatar over a gradient hero, 4x2 action grid, People, Business & bills, Recent payments, floating Scan button
Scan	live camera with Chrome `BarcodeDetector`; fallbacks: gallery photo, typed UPI ID
Pay	big amount, Add a note, account sheet, blue Pay button; UPI cap of 1,00,000 enforced
PIN	6-dot pad, any 4-6 digits (demo)
Success	blue check, payee, UPI id, amount, UPI transaction id, security verdict for heavy payments, Share screenshot, Done
Tiles not built	bank transfer, bills, recharge show a "not available in this demo" snackbar

How it works

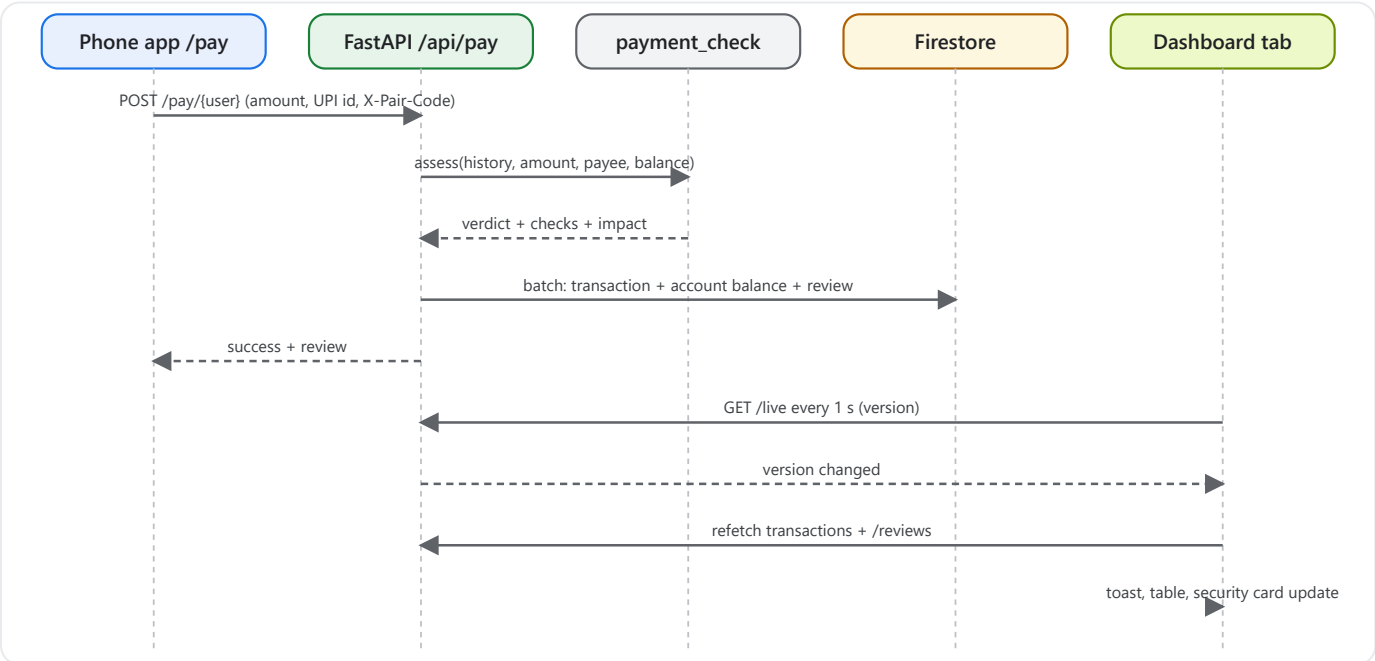


Figure 5. A payment end to end, including how the open dashboard finds out.

- **UPI QR format.** `upi://pay?pa=<vpa>&pn=<name>&am=<amount>&tn=<note>&cu=INR`, parsed by `lib/upi.ts`. Any real UPI QR can be read; the app only reads it and never contacts a bank.

- **Server.** `POST /api/pay/{user}` validates the VPA with a regex and amount `0 < a <= 100000`, categorises the merchant, runs the security review, then writes the transaction, the reduced account balance and the review in **one Firestore batch**.
- **Merchant QR page.** `/pay/qr` on the laptop renders demo merchants with editable amounts for the phone to scan.

Installable app (PWA)

`public/pay.webmanifest` declares `display: standalone`, scope `/pay`, theme blue and 192/512/maskable icons. The page links it through its own `layout.tsx`. Installed from Chrome (Install app) it launches without the address bar. `adb reverse tcp:3000 tcp:3000` and `tcp:8000 tcp:8000` make the phone's `localhost` reach the laptop, which is also a secure context, so the camera works.

Why a web app, not native

One React codebase, instant updates over the dev server, and a PWA is enough to look and feel like an app for this demo. The trade-off is that it depends on the USB forward being active.

Feature 7

Automatic payment security review

Every phone payment is checked as it happens; heavy or unusual ones appear on the dashboard with the verdict and their impact.

What it does

A payment is **heavy** when it is at least ₹25,000, or at least ₹5,000 *and* at least 5 times your median payment. Heavy or non-clean payments store a review that the dashboard shows as a card: verdict badge, the seven checks, four impact tiles, insights, and "It was me" / "I did not make this" buttons.

Decision flow

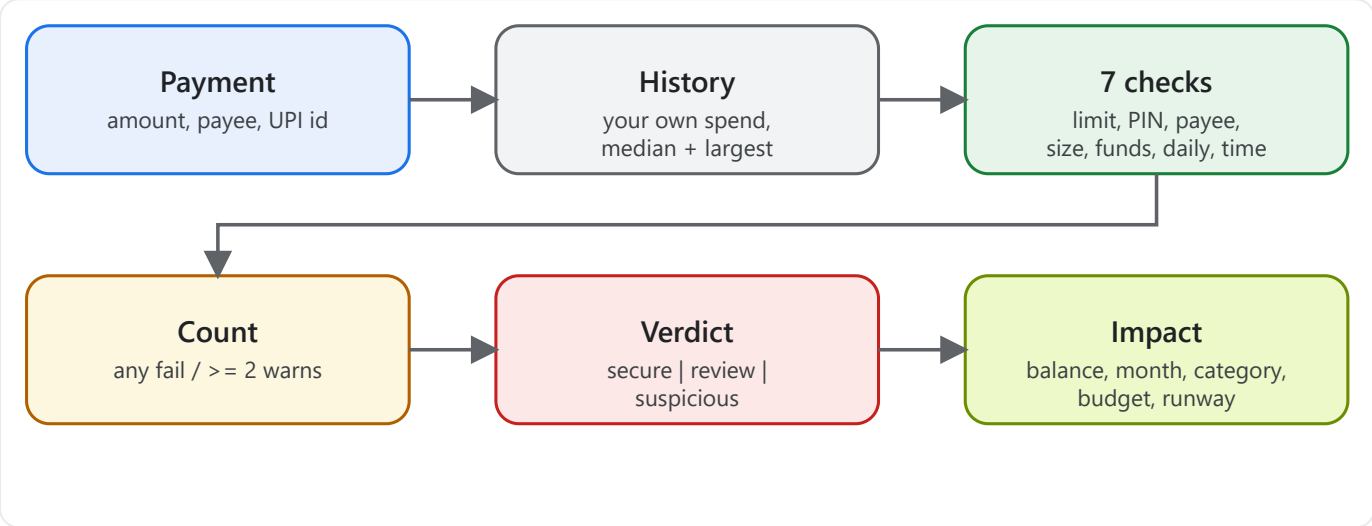


Figure 6. Pure rules over your own history; no model and no network.

The seven checks

Check	Pass when	Warn / fail
Within UPI limit	amount <= ₹1,00,000	fail above
UPI PIN verified	always (PIN entered on device)	-
Payee recognised	you have paid this merchant before	warn: first payment
Size vs your usual	within 10x of your median	warn above 10x
Sufficient balance	balance after is at least one month of essentials	warn below; fail if negative
Daily UPI activity	UPI sent today <= ₹1,00,000	warn above
Time of day	05:00 to 23:00 IST	warn at night

Verdict: any *fail* gives **suspicious**; two or more *warns* give **review**; otherwise **secure**.

Impact analysis

- Balance before and after, and the payment as a percentage of the balance.
- Month-to-date spend after the payment versus an average full month ("30% above your usual month").
- Category spend this month versus its usual, and the share of that category's budget used.
- Runway: balance after divided by typical monthly essentials.

> **Example.** A ₹1,00,000 payment to a new payee from a ₹42,000 balance fails the balance check and is marked **suspicious**, with balance after -₹58,000. A ₹12,000 payment to a known merchant with enough balance comes back **secure**.

Endpoints

Endpoint	Purpose
GET /api/pay/{u}/reviews	newest reviews with checks and impact
POST /api/pay/{u}/reviews/{id}/status	confirmed or reported; reported flags the transaction for review

Why rules, not a model

The user can read exactly why a payment was flagged, behaviour is identical on every run, and it needs no training data. Thresholds are product choices and are shown in each check's detail text.

Feature 8

Phone pairing and live sync

Connect the phone to the account you are signed in with, and see every payment appear in about a second.

The problem it solves

The phone has no login. Without pairing it paid into the demo user, while the website was signed in as a real account, so the two ledgers differed. Pairing makes the phone write to *your* ledger.

Pairing flow

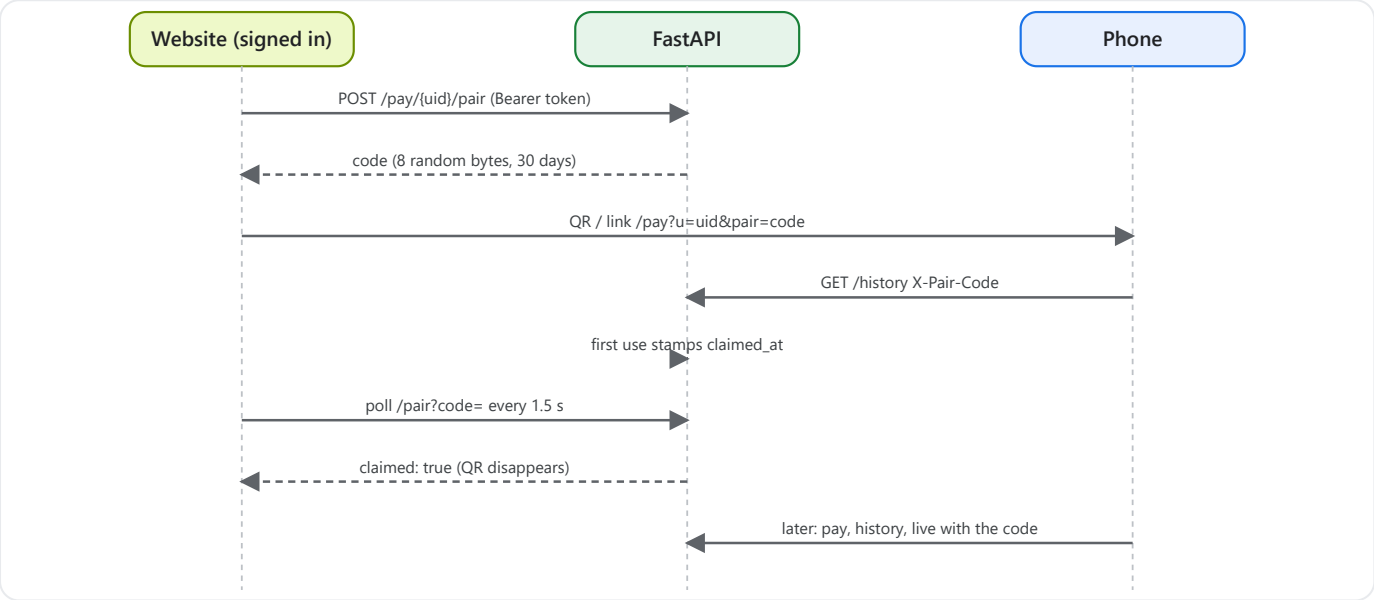


Figure 7. After the first request from the phone, the website replaces the QR with "Phone connected" and returns to the other connectors.

How it works

- **Code.** `secrets.token_urlsafe(8)` stored in `pay_pairs` with the user id; valid 30 days. The server accepts it only for the user that minted it (`403` otherwise).
- **Phone auth.** The phone sends `X-Pair-Code`; a dependency `payer` accepts either that or a normal bearer token. The phone stores `{u, code}` in localStorage so the installed app remembers the account.
- **Live sync.** `LiveSync` (mounted in the app layout) polls `/api/pay/{u}/live` every **1 second**, only while the tab is visible. The server returns a global write counter; when it changes, the API cache is cleared and `DATA_CHANGED_EVENT` makes every mounted `useAsync` refetch. A toast announces new heavy payments.
- **Phone history.** The home screen polls `/history` every 4 seconds from the same ledger.

The Connect page has a **Phone payments** tile next to Google Pay, Sheets and Plaid; the dashboard has the same pairing card.

Why polling, not websockets

A one-line counter endpoint is cheap, works through any proxy and needs no connection state. A websocket or Firestore listener is the upgrade path if many users poll at once.

Limits

The pairing link points at `localhost:3000`, so it only works while the USB forward is active. Pairing a different account means pairing again from that account's session.

Feature 9

Read cache with write-through

Why the Transactions page used to take forever after a payment, and how it stays fast now.

The problem

A cold Firestore read of one account's 281 transactions took about 14 seconds. The old cache dropped **every** cached read after **any** write, so each phone payment sent the dashboard and Transactions page back to cold reads for several queries.

How it works now

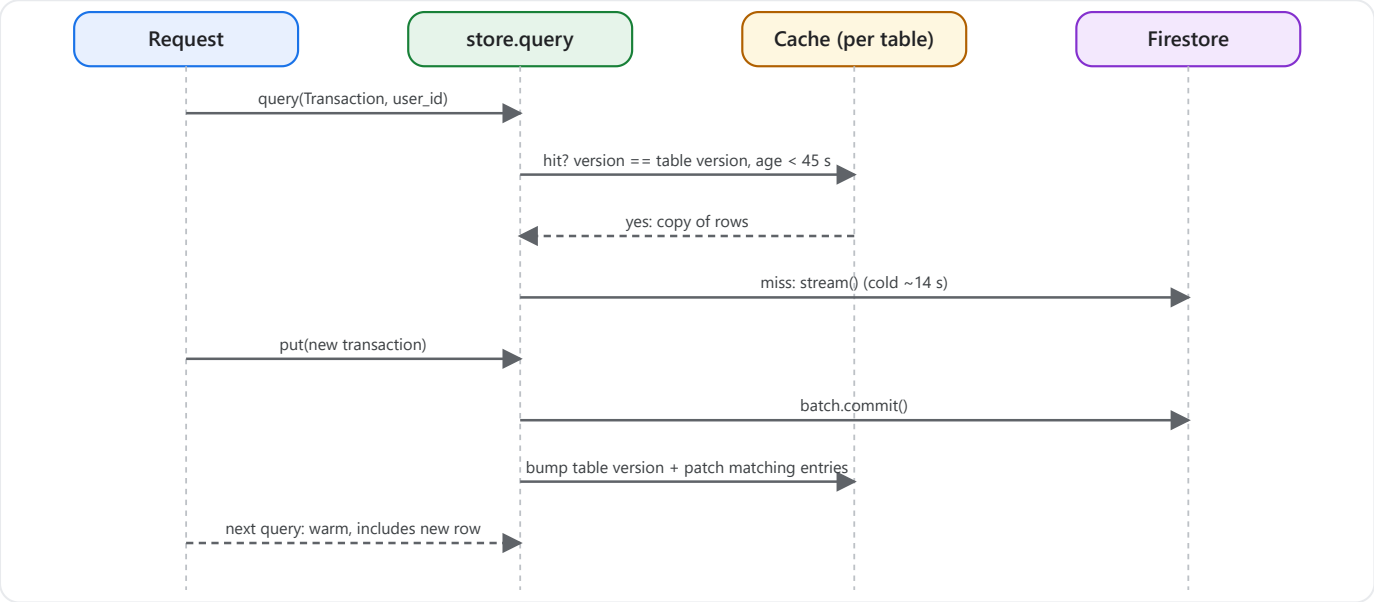


Figure 8. Per-table versions and write-through.

- Each table has its own version number. A write bumps only its table.
- `put()` commits to Firestore, then **patches the still-current cached query results** with the new rows (a row is added to a cached query only if it matches that query's filters), so the next read is warm and already contains the payment.
- A 45 second TTL still covers writes from other processes. Log-like tables (audit, copilot insights) are never cached.
- A separate global counter, bumped on every non-log write, feeds the live-sync endpoint.

Verification

Five tests in `tests/test_read_cache.py`: writes visible immediately, cached rows are copies, repeat reads never touch Firestore, log tables are live, and the new write-through test (no refetch after `put`, filters respected).

Limits

The first read after a backend restart is still cold. Firestore batches are atomic within 500 rows, not across batches.

Feature 10

Login, onboarding and dashboard changes

Smaller changes that make the flow coherent.

Changes

Change	Detail
Redirect	After Google, email or MFA sign-in the app opens `/dashboard` (it used to open the copilot).
Questionnaire	The onboarding wizard opens over the dashboard if the profile is missing or was *skipped*; closing hides it for the browser session, and a skipped profile asks again at the next sign-in.
Status label	Transaction History shows **Completed** for imported rows that were only unreviewed. The money had already moved; review state is tracked in the Transactions review queue.
Security card	Latest heavy or unusual payment, placed at the top of the dashboard.
Phone app look	Full-width layout (a wrapper that shrank to its content made it look squeezed), dev badge hidden in dev.

Where

`app/login/page.tsx`, `components/onboarding/OnboardingGate.tsx`, `components/dashboard/TransactionHistory.tsx`, `PaymentReviewCard.tsx`, `PhonePairCard.tsx`.

Security, privacy and data integrity

What protects the user, and what a reviewer can verify.

Area	Mechanism
Authentication	Firebase ID tokens. `resolve_user_id` requires the token's `sub` to equal the URL user id (403 otherwise). Demo personas and guest sandboxes are public synthetic fixtures.
Phone link	Pairing code is 8 random bytes, bound to one user, expires in 30 days, and only grants the pay, history, live and review endpoints.
Firestore rules	Clients are read-only on their own documents; all writes go through the backend Admin SDK. Unknown collections are denied.
Input validation	Pydantic models: VPA regex, amount bounds, field lengths. Category and review status are validated against enums or regexes.
Privacy of documents	OCR runs locally; identifiers are redacted before preview or storage; the redaction report counts what was removed.
Rate limiting	The voice and AI routes use a sliding-window `RateLimiter`.
Background jobs	Inngest-called stages require a shared secret in `X-Job-Token`, compared with `hmac.compare_digest`.
Integrity (ACID)	A payment's transaction, balance change and review are written in one Firestore batch (atomic, up to 500 writes). Firestore transactions provide isolation and durability. The cache never serves a row older than its write because per-table versions change on commit.
Honest labelling	Demo and synthetic data are labelled; the phone app says no real money moves.

> **Not claimed.** The demo does not talk to a bank or a real UPI rail; PIN entry accepts any 4-6 digits; pairing codes live in the same database as the ledger.

API reference (features in this guide)

Method and path	Auth	Purpose
GET /api/forecast-risk/{u}	bearer / demo	full Forecast & Risk report (params: horizon, sims, seed, profile, source)
POST /api/expenses/{u}/snap	bearer / guest	photo to item and price suggestion
POST /api/expenses/{u}/suggest, /teach	bearer / guest	suggest from name; save a confirmed item and price
POST /api/ocr/extract/{u}, /import/{u}	bearer / demo	OCR + redaction preview; import parsed rows
POST /api/pay/{u}	bearer or X-Pair-Code	book a UPI payment; returns the review
GET /api/pay/{u}/history	bearer or X-Pair-Code	recent UPI payments
GET /api/pay/{u}/live	bearer or X-Pair-Code	global write counter
GET /api/pay/{u}/reviews	bearer or X-Pair-Code	security reviews, newest first
POST /api/pay/{u}/reviews/{id}/status	bearer	confirmed or reported
POST /api/pay/{u}/pair	bearer	mint a pairing code
GET /api/pay/{u}/pair?code=	bearer	has the phone claimed the code / is any phone paired
GET/PUT /api/profile/{u}	bearer / demo	onboarding profile

Testing, running it, and known limits

Verification done

- Backend: 19 Forecast & Risk tests and 5 read-cache tests pass (`python -m pytest --noconftest tests/test_forecast_risk.py tests/test_read_cache.py`).
- Frontend: `tsc --noEmit` and eslint clean on the touched files.
- Manually: heavy payment review verified by API call; pairing claim verified by API (claimed false, then true after first phone request); phone layout checked by screenshots over adb.

Run it

```
cd backend  && python -m uvicorn app.main:app --port 8000
cd frontend && npm run dev            # http://localhost:3000
adb reverse tcp:3000 tcp:3000 && adb reverse tcp:8000 tcp:8000
phone Chrome: http://localhost:3000/pay   (Install app for full screen)
```

Known limits

- **No backend tests yet** for the payment review rules and the pay/pairing endpoints (verified manually and by API calls).
- Dashboard live refresh only runs while the tab is visible; the USB forward drops on unplug.
- Test payments created during development remain in the demo ledger until removed.
- Two different signed-in accounts see two different ledgers; pair the phone from the account you are viewing.
- Backend reload on Windows is unreliable; restart uvicorn after backend edits.

Other modules present in the repo (not covered here)

Voice agent (`api/voice_router.py`), Evidence Council with Inngest jobs (`api/jobs_router.py`, `security/`), Copilot and RAG (`copilot/`, `rag/`), tax, household, goals and budgets. They share the same store, auth and conventions described above.